

LAPORAN PRAKTIKUM MINGGU 7

Topik: Exception Handling, Custom Exception, dan Design Pattern Sederhana

A. IDENTITAS

Nama	:	Sri Wahyuningsih
Nim	:	240202844
Kelas	:	3IKRA

B. TUJUAN

Adapun tujuan dari praktikum ini adalah agar mahasiswa mampu:

1. Memahami perbedaan antara error dan exception dalam Java.
2. Mengimplementasikan mekanisme try-catch-finally dengan benar.
3. Membuat dan menggunakan custom exception sesuai kebutuhan bisnis.
4. Menerapkan exception handling pada studi kasus keranjang belanja (Agri-POS).
5. Menjalankan program Java menggunakan perintah javac dan java.

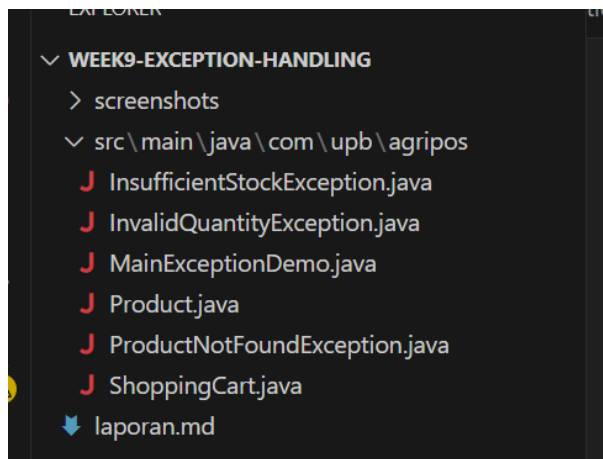
C. DASAR TEORI

Adapun dasar teori yang mendasari praktikum ini, di antaranya:

1. Error merupakan kesalahan fatal yang tidak dapat ditangani oleh program, sedangkan exception adalah kondisi tidak normal yang masih dapat ditangani.
2. Mekanisme try-catch-finally digunakan untuk menangani kesalahan agar program tidak berhenti secara tiba-tiba.
3. Custom exception memungkinkan pengembang mendefinisikan jenis kesalahan bisnis yang lebih spesifik dan informatif.
4. Exception handling sangat penting dalam aplikasi POS untuk menjaga stabilitas sistem.
5. Design pattern sederhana seperti pemisahan Model dan Controller membantu struktur program lebih rapi.

D. LANGKAH PRAKTIKUM

1. Setup Project:
Buat struktur repository seperti berikut:



2. Coding

- a. Membuat class Product sebagai model produk pertanian.
- b. Membuat custom exception:
 - InvalidQuantityException
 - ProductNotFoundException
 - InsufficientStockException
- c. Membuat class ShoppingCart dengan validasi bisnis menggunakan exception.
- d. Membuat class MainExceptionDemo sebagai program utama.
- e. Menguji program menggunakan perintah javac dan java.

3. Commit & Push:

Commit dengan pesan: week9-exception: implement custom exception pada shopping cart

E. KODE PROGRAM

1. Product.java

```

src > main > java > com > upb > agripos > Product.java
1  package com.upb.agripos;
2
3  public class Product {
4
5      private final String code;
6      private final String name;
7      private final double price;
8      private int stock;
9
10     public Product(String code, String name, double price, int stock) {
11         this.code = code;
12         this.name = name;
13         this.price = price;
14         this.stock = stock;
15     }
16
17     public String getCode() {
18         return code;
19     }
20
21     public String getName() {
22         return name;
23     }

```

```

23     }
24
25     public double getPrice() {
26         return price;
27     }
28
29     public int getStock() {
30         return stock;
31     }
32
33     public void reduceStock(int qty) {
34         this.stock -= qty;
35     }
36 }
37

```

2. InvalidQuantityException.java

```

src > main > java > com > upb > agripas > InvalidQuantityException.java > ...
1  package com.upb.agripas;
2
3  public class InvalidQuantityException extends Exception {
4      public InvalidQuantityException(String msg) {
5          super(msg);
6      }
7  }
8

```

3. ProductNotFoundException.java

```

src > main > java > com > upb > agripas > ProductNotFoundException.java > ...
1  package com.upb.agripas;
2
3  public class ProductNotFoundException extends Exception {
4      public ProductNotFoundException(String msg) {
5          super(msg);
6      }
7  }
8

```

4. InsufficientStockException.java

```

src > main > java > com > upb > agripas > InsufficientStockException.java > ...
1  package com.upb.agripas;
2
3  public class InsufficientStockException extends Exception {
4      public InsufficientStockException(String msg) {
5          super(msg);
6      }
7  }
8

```

5. ShoppingCart.java

```

J ShoppingCart.java X J MainExceptionDemo.java J InvalidQuantityException.class J ProductNotFoundException
src > main > java > com > upb > agripos > J ShoppingCart.java > ...
1 package com.upb.agripos;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class ShoppingCart {
7
8     private final Map<Product, Integer> items = new HashMap<>();
9
10    // tambah produk ke keranjang
11    public void addProduct(Product p, int qty) throws InvalidQuantityException {
12        if (qty <= 0) {
13            throw new InvalidQuantityException(
14                msg: "Quantity harus lebih dari 0."
15            );
16        }
17
18        items.put(p, items.getOrDefault(p, defaultValue: 0) + qty);
19    }
20
21    // hapus produk dari keranjang
22    public void removeProduct(Product p) throws ProductNotFoundException {
23        if (!items.containsKey(p)) {
24            throw new ProductNotFoundException(
25                msg: "Produk tidak ada dalam keranjang."
26            );
27        }
28    }

```

```

28
29        items.remove(p);
30    }
31
32    // proses checkout
33    public void checkout() throws InsufficientStockException {
34
35        // validasi stok
36        for (Map.Entry<Product, Integer> entry : items.entrySet()) {
37            Product product = entry.getKey();
38            int qty = entry.getValue();
39
40            if (product.getStock() < qty) {
41                throw new InsufficientStockException(
42                    "Stok tidak cukup untuk produk: " + product.getName()
43                );
44            }
45        }
46
47        // pengurangan stok (jika semua valid)
48        for (Map.Entry<Product, Integer> entry : items.entrySet()) {
49            entry.getKey().reduceStock(entry.getValue());
50        }
51    }
52
53

```

6. MainExceptionDemo.java

```

J ShoppingCart.java X J MainExceptionDemo.java X J InvalidQuantityException.class J ProductNotFoundException
src > main > java > com > upb > agripos > J MainExceptionDemo.java > MainExceptionDemo > main
1 package com.upb.agripos;
2
3 public class MainExceptionDemo {
4
5     public static void main(String[] args) {
6
7         System.out.println(x: "Hello, I am Sriwa-240202844 (Week9)");
8
9         ShoppingCart cart = new ShoppingCart();
10
11         Product sekop = new Product(
12             code: "SRW-025",
13             name: "Sekop Tangan",
14             price: 50000,
15             stock: 5
16         );
17
18         // Uji quantity tidak valid
19         try {
20             cart.addProduct(sekop, -1);
21         } catch (InvalidQuantityException e) {
22             System.out.println("Kesalahan: " + e.getMessage());
23         } finally {
24             System.out.println(x: "Validasi quantity selesai.\n");
25         }
26

```

```

25     }
26
27     // Uji hapus produk yang tidak ada di keranjang
28     try {
29         cart.removeProduct(sekop);
30     } catch (ProductNotFoundException e) {
31         System.out.println("Kesalahan: " + e.getMessage());
32     }
33
34     // Uji stok tidak cukup saat checkout
35     try {
36         cart.addProduct(sekop, qty: 10); // lebih besar dari stok (5)
37         cart.checkout();
38     } catch (InvalidQuantityException | InsufficientStockException e) {
39         System.out.println("Kesalahan: " + e.getMessage());
40     }
41
42     System.out.println(x: "\nProgram selesai.");
43 }
44
45

```

F. HASIL EKSEKUSI

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Run: MainExceptionDem

PS C:\Users\sri61\Documents\oop-pos-240202844\praktikum\week9-exception-handling> & 'C:\Program
a.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\sri61\AppData\Roaming\Code\User
8dcd496a6203280b4627b0b2\redhat.java\jdt_ws\week9-exception-handling_88fe4cb\bin' 'com.upb.agripo
Hello, I am Sriwa-240202844 (Week9)
Kesalahan: Quantity harus lebih dari 0.
Validasi quantity selesai.

Kesalahan: Produk tidak ada dalam keranjang.
Kesalahan: Stok tidak cukup untuk produk: Sekop Tangan

Program selesai.
PS C:\Users\sri61\Documents\oop-pos-240202844\praktikum\week9-exception-handling>

```

G. ANALISIS

Program berhasil menerapkan exception handling untuk menangani kesalahan bisnis pada sistem Agri-POS.

Penggunaan custom exception membuat pesan kesalahan lebih jelas dan mudah dipahami. Dengan adanya try-catch-finally, program tetap stabil meskipun terjadi kesalahan input. Pendekatan ini membuat sistem lebih aman dan realistis untuk aplikasi POS.

H. KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa:

- Exception handling sangat penting untuk menjaga kestabilan program.
- Custom exception memudahkan identifikasi kesalahan bisnis.
- Mekanisme try-catch-finally membuat program lebih robust.
- Implementasi exception handling relevan untuk sistem Agri-POS.

QUIZ

1. Jelaskan perbedaan error dan exception.

Jawaban:

Error adalah kesalahan serius yang bersifat fatal dan umumnya tidak dapat ditangani oleh program, seperti OutOfMemoryError.

Exception adalah kondisi tidak normal yang terjadi saat program berjalan dan masih

dapat ditangani menggunakan mekanisme try–catch, misalnya kesalahan input atau data tidak valid.

2. Apa fungsi finally dalam blok try–catch–finally?

Jawaban:

Blok finally digunakan untuk menjalankan kode yang selalu dieksekusi, baik terjadi exception maupun tidak.

Biasanya digunakan untuk proses pembersihan sumber daya atau menampilkan pesan bahwa proses telah selesai.

3. Mengapa custom exception diperlukan?

Jawaban:

Custom exception diperlukan agar kesalahan logika bisnis dapat ditangani secara lebih spesifik dan pesan kesalahan menjadi lebih jelas serta mudah dipahami, dibandingkan menggunakan exception umum bawaan Java.

4. Berikan contoh kasus bisnis dalam POS yang membutuhkan custom exception.

Jawaban:

Contoh kasus dalam sistem POS yang membutuhkan custom exception antara lain:

- Jumlah pembelian tidak valid ($\text{quantity} \leq 0$).
- Produk tidak ditemukan di dalam keranjang belanja.
- Stok produk tidak mencukupi saat proses checkout.

CHECKLIST KEBERHASILAN

- | |
|--|
| <ul style="list-style-type: none">✓ Mahasiswa mampu menjelaskan error vs exception✓ Program menggunakan try–catch–finally✓ Minimal dua custom exception berhasil dibuat✓ Exception berhasil terintegrasi dalam keranjang belanja✓ Laporan lengkap dan commit benar |
|--|